

ANURAG NAVIN BHANDARY

+91 8169370520 | bhandaryanurag@gmail.com | anuragbhandary.github.io | linkedin.com/in/bhandary-anurag | github.com/AnuragBhandary | Mumbai, India

SUMMARY

Software developer with a year of production experience on the founding team of an early-stage AI sports startup, working on **asynchronous services**, **event-driven pipelines**, and applied **LLM** systems. Primary stack is **Python** with **FastAPI**, **asyncio**, **Kafka**, **Redis**, **PostgreSQL**, and **Docker**, with an emphasis on throughput, ordering guarantees, and reliable failure recovery.

TECHNICAL SKILLS

Languages: Python (primary), SQL, Java (core language, data structures and algorithms), JavaScript

Backend and APIs: FastAPI, REST APIs, WebSockets, API-key auth, rate limiting

Distributed Systems: Apache Kafka, Redis (pub/sub, queues), async workers, **idempotency**, retries, backpressure, event replay

Data and Cloud: PostgreSQL, AWS S3, Docker, Docker Compose

AI and Media: Gemini, Groq, ElevenLabs, structured LLM output, OCR, ffmpeg

EXPERIENCE

Software Developer (Founding Team) | Fayble Inc. | Remote | Jan 2026 - Jun 2026

Software Development Intern | Fayble Inc. | Remote | Jul 2025 - Dec 2025

AI sports-simulation platform

- Engineered the **audio pipeline** for a real-time AI sports-commentary system, generating **dual-commentator** speech with **ElevenLabs** and stitching per-play clips into continuous broadcast output.
- Benchmarked **Gemini** against **Groq**, measured roughly **10x lower** text-generation latency with Groq, and designed a single-emotion-per-play workflow that cut audio-generation time about **threefold**.
- Automated archery match production with **ffmpeg**, synchronizing AI commentary, crossfades, multi-channel audio, and structured per-play metadata into finished match videos.
- Integrated **Apache Kafka** into the platform's animation flow, replaying **19 games** and roughly **1,274 Kafka messages** through an ordered ingestion path that blocked duplicates, then served live game state over **FastAPI** and **WebSockets**.

PROJECTS

Distributed Task Scheduler and Job Queue | Python, FastAPI, Redis, PostgreSQL, Celery-style API, Docker

- Designed a multi-service job platform with scheduled and immediate execution, **exponential-backoff retries**, dead-letter handling, **worker leases** with fencing tokens, **idempotency keys**, and persisted execution history.
- Sustained **2,000 jobs per minute** over a 50,000-job run while a worker was killed every minute: **zero** unrecovered failures, **p95 enqueue latency of 7 ms**, and no duplicate side effects across retries and crash recovery.
- Shipped REST endpoints and a **Celery-style Python client** with API-key auth and rate limiting; containerized API, scheduler, workers, Redis, and PostgreSQL at **94% test coverage**.

Real-Time Event Streaming Platform | Python, asyncio, FastAPI, WebSockets, Redis, PostgreSQL

- Wrote an **asyncio** event service that persists ordered events, serves state snapshots, and fans live updates across three instances through **FastAPI**, **WebSockets**, and **Redis pub/sub**.
- Held **p95 delivery latency at 5 ms** with **1,000 concurrent WebSocket clients** and 100,000 events (**5 million deliveries**), preserving monotonic sequence numbers.
- Added cursor-based reconnection, bounded replay, heartbeats, slow-consumer handling, and **backpressure**, verifying zero duplicate or out-of-order events across **14,000+ reconnects** under client drops and instance kills.

Streaming Consumer Benchmark: asyncio vs Virtual Threads | Java 21, Python asyncio, Kafka, Docker, JMH

- Reimplemented the fan-out consumer hot path from the streaming platform in **Java 21**, holding up to **20,000 socket connections** on **virtual threads** under a blocking programming model rather than an event loop.
- Compared both through one load harness on identical workloads up to **320,000 deliveries/s**, tracking **p99 latency**; traced a dispatch bottleneck and cut 20k-connection p50 from **87 ms to 2.3 ms**.
- Controlled for **JVM warmup** by discarding early iterations per run, and documented where the two concurrency models diverge under slow-consumer and **backpressure** conditions.

EDUCATION

M.S. Computer Science | University of Texas at Arlington | Aug 2023 - May 2025

GPA 3.42/4.0 | Focus: Artificial Intelligence and Software Engineering | Arlington, TX

B.E. Electronics Engineering | University of Mumbai | Aug 2019 - May 2023

GPA 8.36/10 | Mumbai, India

Competitive Coding: LeetCode 300+ problems solved in Python and Java across data structures and algorithms |

leetcode.com/anuragb2901